

Buchtausch-App

Data-Mart-Erstellung in SQL (DLBDSPBDM01_D)

Erarbeitungs- und Reflexionsphase

Agenda

1. Projektkontext
2. Datenbankimplementierung
3. Zusammenfassung

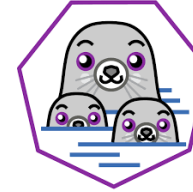
1. Projektkontext

Technisches Umfeld, App-Module



Oracle Database

- Oracle AI Database 26ai
Free Release 23.26.1.0.0



Podman

- Container-Runtime zur
Bereitstellung der
Oracle-Instanz
- Version 5.8.2



Oracle SQL Developer

- Ausführung und Test
der SQL-Statements
- Version 24.3.1.347



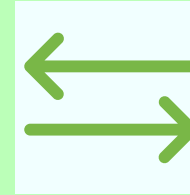
DBeaver

- Erstellung des ER-
Diagramms
- Version 26.0.0



Katalog

- Buch anlegen
- Exemplar einstellen
- Suchen
- Duplikat melden



Ausleihprozess

- Anfrage stellen
- Bestätigen / Ablehnen
- Ausleihen
- Zurückgeben



Bewertungen

- Buch bewerten
- Nutzer bewerten



Benutzer

- Registrieren
- Anmelden
- Profil bearbeiten
- Rollen zuweisen

2. Datenbankimplementierung

ER-Diagramm, DDL, SQL-Statements

ER-Diagramm

Legende

Katalog

Ausleihprozess

Bewertungen

Benutzer

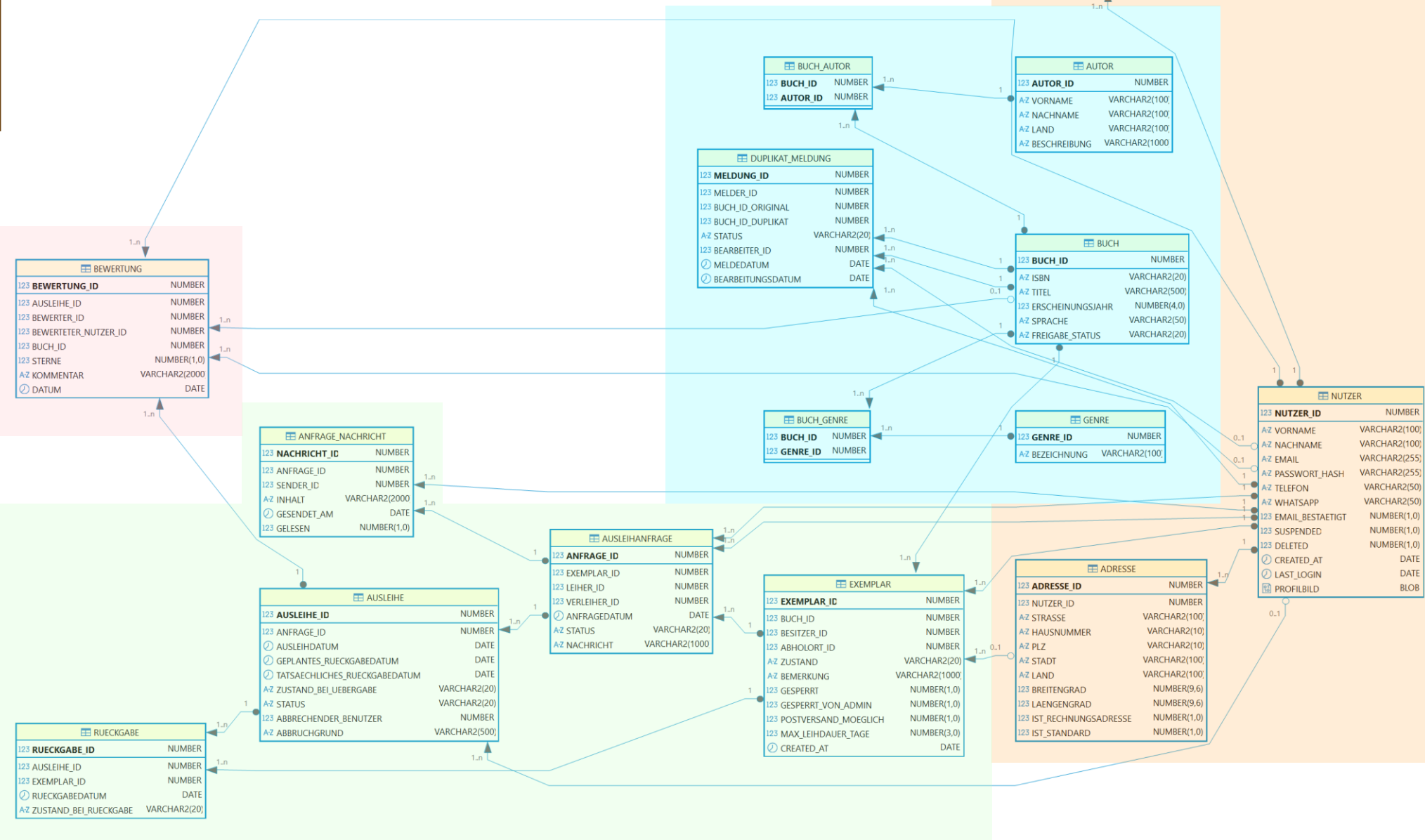


Tabelle: Buch

```
CREATE SEQUENCE seq_buch START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE buch (  
  buch_id      NUMBER      DEFAULT seq_buch.NEXTVAL PRIMARY KEY,  
  isbn         VARCHAR2(20) UNIQUE,  
  titel        VARCHAR2(500) NOT NULL,  
  erscheinungsjahr  NUMBER(4),  
  sprache      VARCHAR2(50)  DEFAULT 'Deutsch' NOT NULL,  
  freigabe_status  VARCHAR2(20)  DEFAULT 'ENTWURF' NOT NULL  
);  
  
SELECT      b.titel,  
            a.vorname || ' ' || a.nachname      AS autor,  
            g.bezeichnung                        AS genre,  
            b.sprache,  
            b.erscheinungsjahr  
  
FROM        buch b  
JOIN        buch_autor ba  ON ba.buch_id = b.buch_id  
JOIN        autor a        ON a.autor_id = ba.autor_id  
JOIN        buch_genre bg  ON bg.buch_id = b.buch_id  
JOIN        genre g        ON g.genre_id = bg.genre_id  
WHERE       ba.autor_id = (  
            SELECT      MIN(ba2.autor_id)  
            FROM        buch_autor ba2  
            WHERE       ba2.buch_id = b.buch_id  
            )  
AND         bg.genre_id = (  
            SELECT      MIN(bg2.genre_id)  
            FROM        buch_genre bg2  
            WHERE       bg2.buch_id = b.buch_id  
            )  
ORDER BY    b.titel;
```



Bücher

Die Tabelle fasst die Metadaten eines Buches zusammen, die unabhängig von seinen physischen Exemplaren sind.

Autor und Genre sind keine Attribute des Buches, sondern werden in eigenen Tabellen geführt. Über diese Felder kann ein Nutzer aber nach Büchern suchen.

Die Query liefert alle Bücher mit ihrem ersten Autor und ersten Genre.

↕ TITEL	↕ AUTOR	↕ GENRE	↕ SPRACHE	↕ ERSCHEINUNGSJAHR
1 三体	Liu Cixin	Fiction	Deutsch	2008
2 Записки изъ подполья	Fyodor Dostoyevsky	Fiction	Deutsch	1864
3 Мастер и Маргарита	Mikhail Bulgakov	Fiction	Deutsch	1966
4 Понедельник начинается в субботу; Пикник на обочине	Arkady Strugatsky	Fiction	Englisch	1978
5 A Clash of Kings	George R. R. Martin	Fiction	Englisch	1998
6 A Closed and Common Orbit	Becky Chambers	Fiction	Deutsch	2016
7 A Court of Silver Flames	Sarah J. Maas	Fiction	Deutsch	2021
8 A Darker Shade of Magic	V. E. Schwab	Fiction	Englisch	2015
9 A Deepness in the Sky (Zones of Thought)	Vernor Vinge	Fiction	Englisch	1999
10 A Fire upon the Deep	Vernor Vinge	Fiction	Englisch	1992
11 A Game of Thrones	George R. R. Martin	Fiction	Deutsch	1996
12 A Gentleman in Moscow	Aman Toules	Fiction	Deutsch	2018

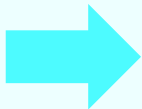
Teilmenge,
insgesamt 300 Datensätze

Tabelle: Autor

```
CREATE SEQUENCE seq_autor START WITH 1000 INCREMENT BY 1 NOCACHE;

CREATE TABLE autor (
  autor_id    NUMBER      DEFAULT seq_autor.NEXTVAL PRIMARY KEY,
  vorname     VARCHAR2(100),
  nachname    VARCHAR2(100) NOT NULL,
  land        VARCHAR2(100),
  beschreibung VARCHAR2(1000)
);

SELECT      a.vorname || ' ' || a.nachname      AS autor,
            COUNT(ba.buch_id)                  AS anzahl_buecher
FROM        autor a
LEFT JOIN   buch_autor ba ON ba.autor_id = a.autor_id
WHERE a.land like 'United Kingdo%'
GROUP BY    a.vorname, a.nachname
ORDER BY    anzahl_buecher DESC;
```



AUTOR	ANZAHL_BUECHER
1 Agatha Christie	21
2 Terry Pratchett	11
3 Douglas Adams	4
4 J. R. R. Tolkien	4
5 H. G. Wells	3
6 Piers Anthony	3
7 Neil Gaiman	3
8 Joe Abercrombie	3
9 C. S. Lewis	2
10 Frances Hodgson Burnett	2
11 Jane Austen	2
12 George Orwell	2
13 Arthur C. Clarke	2
14 Richard Adams	1
15 Oscar Wilde	1
16 J. K. Rowling	1

Teilmenge,
insgesamt 27 Datensätze

Autoren

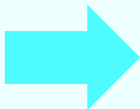
Autoren werden unabhängig von Büchern geführt.

Land und Beschreibung ermöglichen die Unterscheidung gleichnamiger Autoren.

Die Query zeigt alle Autoren aus UK mit der Anzahl ihrer Bücher im Katalog.

Tabelle: Buch_autor

```
CREATE TABLE buch_autor (  
  buch_id  NUMBER NOT NULL REFERENCES buch(buch_id),  
  autor_id NUMBER NOT NULL REFERENCES autor(autor_id),  
  CONSTRAINT pk_buch_autor PRIMARY KEY (buch_id, autor_id)  
);  
  
SELECT      b.titel,  
            COUNT(ba.autor_id)          AS anzahl_autoren  
FROM        buch_autor ba  
JOIN        buch b          ON b.buch_id = ba.buch_id  
GROUP BY    b.titel  
HAVING      COUNT(ba.autor_id) > 1  
ORDER BY    anzahl_autoren DESC;
```



TITEL	ANZAHL_AUTOREN
1 Atlas Shrugged (Centennial Ed. HC)	5
2 Pollyanna	4
3 三体	4
4 The Outsiders	3
5 Dragons of Winter Night	3
6 The Sun Also Rises	3
7 The Mote in God's Eye	3
8 The Day of the Triffids	3
9 The Difference Engine	2
10 Diary of a Wimpy Kid	2
11 The Light Fantastic	2
12 Good Omens	2
13 The Poppy War	2
14 Czas pogardy	2
15 Enchanters' End Game (The Belgariad)	2
16 This is How You Lose the Time War	2
17 Kindred	2
18 Small Gods	2
19 Dawn	2
20 Понедельник начинается в субботу; Пикник на обочине	2

Teilmenge,
insgesamt 25 Datensätze

Buch-Autoren-Zuordnung

Jeder Autor kann mehrere Bücher schreiben, aber jedes Buch kann auch von mehreren Autoren geschrieben worden sein (n:m-Beziehung)

Die Abfrage zeigt Bücher, die von mehreren Autoren geschrieben wurden.

Tabelle: Genre

```
CREATE SEQUENCE seq_genre START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE genre (  
  genre_id  NUMBER      DEFAULT seq_genre.NEXTVAL PRIMARY KEY,  
  bezeichnung VARCHAR2(100) NOT NULL UNIQUE  
);
```

```
SELECT g.bezeichnung, COUNT(bg.buch_id) AS anzahl_buecher  
FROM genre g  
LEFT JOIN buch_genre bg ON bg.genre_id = g.genre_id  
GROUP BY g.bezeichnung  
ORDER BY anzahl_buecher DESC;
```



	BEZEICHNUNG	ANZAHL_BUECHER
1	Fantasy	151
2	Science Fiction	149
3	Adventure	83
4	Children	75
5	Mystery	48
6	Romance	39
7	History	36
8	Horror	32
9	Classics	25
10	Young Adult	23
11	Thriller	13
12	Sonstiges	3
13	Biography	1

Genres

Genres kategorisieren Bücher thematisch.

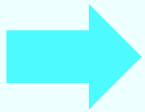
Jedem Buch kann über die Kreuztabelle BUCH_GENRE ein oder mehrere Genres zugeordnet werden (n:m).

Die Abfrage zeigt die Anzahl der Bücher je Genre.

Tabelle: Buch_genre

```
CREATE TABLE buch_genre (  
  buch_id  NUMBER NOT NULL REFERENCES buch(buch_id),  
  genre_id NUMBER NOT NULL REFERENCES genre(genre_id),  
  CONSTRAINT pk_buch_genre PRIMARY KEY (buch_id, genre_id)  
);
```

```
SELECT anzahl_genres, COUNT(*) AS anzahl_buecher  
FROM (  
  SELECT buch_id, COUNT(genre_id) AS anzahl_genres  
  FROM buch_genre  
  GROUP BY buch_id  
)  
GROUP BY anzahl_genres  
ORDER BY anzahl_genres;
```



ANZAHL_GENRES	ANZAHL_BUECHER
1	109
2	87
3	57
4	24
5	14
6	5
7	4

Buch-Genre-Zuordnung

BUCH_GENRE bildet die n:m-Beziehung zwischen Büchern und Genres ab.

Die Abfrage zeigt, wie viele Bücher einem, zwei oder mehr Genres zugeordnet sind.

Nur etwa ein Drittel der Bücher lässt sich exakt einem Genre zuordnen. Einige Bücher haben sogar sechs oder sieben Genres.

Tabelle: Duplikat_meldung

```
CREATE SEQUENCE seq_duplikat_meldung START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE duplikat_meldung (  
  meldung_id      NUMBER      DEFAULT seq_duplikat_meldung.NEXTVAL PRIMARY KEY,  
  melder_id       NUMBER      NOT NULL REFERENCES nutzer(nutzer_id),  
  buch_id_original NUMBER      NOT NULL REFERENCES buch(buch_id),  
  buch_id_duplikat NUMBER      NOT NULL REFERENCES buch(buch_id),  
  status          VARCHAR2(20) DEFAULT 'OFFEN' NOT NULL  
                      CHECK (status IN ('OFFEN','BEARBEITET','ABGELEHNT')),  
  bearbeiter_id   NUMBER      REFERENCES nutzer(nutzer_id),  
  meldedatum      DATE        DEFAULT SYSDATE NOT NULL,  
  bearbeitungsdatum DATE  
);
```

```
SELECT dm.meldung_id,  
       dm.meldedatum,  
       b1.titel AS original,  
       b2.titel AS duplikat,  
       n.vorname || ' ' || n.nachname AS gemeldet_von  
FROM   duplikat_meldung dm  
JOIN   buch b1  ON b1.buch_id = dm.buch_id_original  
JOIN   buch b2  ON b2.buch_id = dm.buch_id_duplikat  
JOIN   nutzer n  ON n.nutzer_id = dm.melder_id  
WHERE  dm.status = 'OFFEN'  
ORDER BY dm.meldedatum;
```



	⚡ MELDUNG_ID	⚡ MELEDATUM	⚡ ORIGINAL	⚡ DUPLIKAT	⚡ GEMELDET_VON
1	1002	03.02.26	The Outsiders	Belgarath the sorcerer	Nico Richter
2	1003	11.02.26	The Cruel Prince	The Mirror Crack'd from Side to Side	Viktor Krüger
3	1005	05.03.26	Coraline	The Road to Oz	Robert Neumann
4	1007	02.04.26	Misery	Oliver Twist	Yvonne Schmitt
5	1009	20.05.26	The Caves of Steel	Dracula	Karin Schäfer

Meldung von Duplikaten

Nutzer können Bucheinträge melden, die sie für Duplikate halten.

Ein Stammdatenpfleger prüft die Meldung und setzt den Status auf BEARBEITET oder ABGELEHNT.

Die Abfrage listet alle offenen Meldungen mit den betroffenen Buchtiteln und dem meldenden Nutzer.

Die Testdaten enthalten allerdings keine echten Duplikate, die Buch-IDs sind willkürlich gewählt.

Tabelle: Exemplar

```
CREATE SEQUENCE seq_exemplar START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE exemplar (  
  exemplar_id      NUMBER      DEFAULT seq_exemplar.NEXTVAL PRIMARY KEY,  
  buch_id          NUMBER      NOT NULL REFERENCES buch(buch_id),  
  besitzer_id      NUMBER      NOT NULL REFERENCES nutzer(nutzer_id),  
  abholort_id      NUMBER      REFERENCES adresse(adresse_id),  
  zustand          VARCHAR2(20) NOT NULL CHECK (zustand IN  
( 'NEU', 'GUT', 'AKZEPTABEL', 'SCHLECHT' )),  
  bemerkung        VARCHAR2(1000),  
  gesperrt         NUMBER(1)    DEFAULT 0 NOT NULL CHECK (gesperrt IN (0,1)),  
  postversand_moeglich NUMBER(1) DEFAULT 0 NOT NULL CHECK  
(postversand_moeglich IN (0,1)),  
  max_leihdauer_tage  NUMBER(3),  
  created_at        DATE        DEFAULT SYSDATE NOT NULL  
);
```

```
SELECT b.titel, u.vorname, u.nachname, e.max_leihdauer_tage, e.zustand  
FROM exemplar e  
JOIN buch b ON b.buch_id = e.buch_id  
JOIN buch_genre bg ON bg.buch_id = b.buch_id  
JOIN genre g ON g.genre_id = bg.genre_id  
JOIN nutzer u ON u.nutzer_id = e.besitzer_id  
WHERE g.bezeichnung = 'Romance'  
AND e.postversand_moeglich = 1  
ORDER BY b.titel, e.zustand, e.max_leihdauer_tage;
```



	TITEL	VORNAME	NACHNAME	MAX_LEIHDAUER_TAGE	ZUSTAND
1	A Princess of Mars	Greta	Wagner	21	AKZEPTABEL
2	Alas de ónix	Thomas	Zimmermann	14	GUT
3	All your perfects	Ursula	Braun	21	GUT
4	Anne of the Island	Felix	Meyer	21	AKZEPTABEL
5	House of Earth and Blood	Olga	Klein	14	GUT
6	It Ends With Us	Clara	Schneider	30	NEU
7	La Peste	Hans	Becker	21	GUT
8	Mrs. Dalloway	Ina	Schulz	21	GUT
9	Phantastes	Anna	Müller	21	GUT
10	Powerless	Clara	Schneider	14	GUT
11	Pride and Prejudice	Lars	Koch	14	GUT
12	Red, White & Royal Blue	David	Fischer	21	GUT
13	Red, White & Royal Blue	Queenie	Schröder	14	NEU
14	Restore me	Olga	Klein	21	GUT
15	Shatter Me	Anna	Müller	30	NEU

Teilmenge,
insgesamt 24 Datensätze

Exemplar

Exemplare sind die physischen Bücher im Bestand der Nutzer.

Neben Zustand und maximaler Leihdauer kann ein Exemplar als postversandfähig markiert sein.

Die Abfrage liefert alle Romance-Exemplare, die per Post versendet werden können, mit Besitzer und Konditionen.

Tabelle: Ausleihanfrage

```
CREATE SEQUENCE seq_ausleihanfrage START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE ausleihanfrage (  
  anfrage_id  NUMBER      DEFAULT seq_ausleihanfrage.NEXTVAL PRIMARY KEY,  
  exemplar_id NUMBER      NOT NULL REFERENCES exemplar(exemplar_id),  
  leiher_id   NUMBER      NOT NULL REFERENCES nutzer(nutzer_id),  
  verleih_id  NUMBER      NOT NULL REFERENCES nutzer(nutzer_id),  
  anfragedatum DATE       DEFAULT SYSDATE NOT NULL,  
  status      VARCHAR2(20) DEFAULT 'OFFEN' NOT NULL  
    CHECK (status IN ('OFFEN','BESTAETIGT','ABGELEHNT','STORNIERT')),  
  nachricht   VARCHAR2(1000)  
);
```

```
SELECT a.anfrage_id, a.anfragedatum, b.titel,  
       n.vorname || ' ' || n.nachname AS leiher,  
       a.status  
FROM   ausleihanfrage a  
JOIN   exemplar e ON e.exemplar_id = a.exemplar_id  
JOIN   buch b ON b.buch_id = e.buch_id  
JOIN   nutzer n ON n.nutzer_id = a.leiher_id  
WHERE  a.verleiher_id = 25  
ORDER BY a.anfragedatum;
```



	ANFRAGE_ID	ANFRAGEDATUM	TITEL	LEIHER	STATUS
1	2010	20.03.26	The Caves of Steel	Jan Hoffmann	OFFEN
2	2014	05.05.26	Small Gods	Robert Neumann	OFFEN

Ausleihanfragen

Eine Ausleihanfrage stellt ein Nutzer, wenn er ein Exemplar ausleihen möchte.

Der Verleiher kann sie bestätigen, ablehnen oder ignorieren.

Die Abfrage zeigt die Anfragen aus der Sicht des Verleihers mit der Benutzer-Id 25.

Tabelle: Ausleihe

```
CREATE SEQUENCE seq_ausleihe START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE ausleihe (
    ausleihe_id          NUMBER      DEFAULT seq_ausleihe.NEXTVAL PRIMARY KEY,
    anfrage_id           NUMBER      NOT NULL UNIQUE REFERENCES
ausleihanfrage(anfrage_id),
    ausleihdatum         DATE        DEFAULT SYSDATE NOT NULL,
    geplantes_rueckgabedatum DATE    NOT NULL,
    tatsaechliches_rueckgabedatum DATE,
    zustand_bei_uebergabe VARCHAR2(20) NOT NULL CHECK (zustand_bei_uebergabe IN
('NEU','GUT','AKZEPTABEL','SCHLECHT')),
    status               VARCHAR2(20) DEFAULT 'LAUFEND' NOT NULL CHECK (status IN
('LAUFEND','UEBERGEBEN','ABGESCHLOSSEN','ABGEBROCHEN')),
    abbrechender_benutzer NUMBER      REFERENCES nutzer(nutzer_id),
    abbruchgrund          VARCHAR2(500)
);
```

```
SELECT a.ausleihe_id, b.titel,
       n.vorname || ' ' || n.nachname AS leiher,
       a.geplantes_rueckgabedatum,
       SYSDATE - a.geplantes_rueckgabedatum AS tage_ueberfaellig
FROM ausleihe a
JOIN anfrage an ON an.anfrage_id = a.anfrage_id
JOIN exemplar e ON e.exemplar_id = an.exemplar_id
JOIN buch b ON b.buch_id = e.buch_id
JOIN nutzer n ON n.nutzer_id = an.leiher_id
WHERE a.status = 'LAUFEND'
AND a.geplantes_rueckgabedatum < SYSDATE
ORDER BY tage_ueberfaellig DESC;
```

Ausleihen

Ein Ausleihe-Datensatz wird angelegt,
wenn eine Anfrage bestätigt wird.

Der Status wechselt auf
ABGESCHLOSSEN, sobald das
Exemplar zurückgegeben wurde.

Die Abfrage zeigt laufende Ausleihen, deren Rückgabedatum überschritten ist, um beispielsweise automatische Mahnungen zu versenden.

[illegible]

Tabelle: Rueckgabe

```
CREATE SEQUENCE seq_rueckgabe START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE rueckgabe (  
  rueckgabe_id      NUMBER      DEFAULT seq_rueckgabe.NEXTVAL PRIMARY KEY,  
  ausleihe_id       NUMBER      NOT NULL UNIQUE REFERENCES ausleihe(ausleihe_id),  
  exemplar_id       NUMBER      NOT NULL REFERENCES exemplar(exemplar_id),  
  rueckgabedatum    DATE        DEFAULT SYSDATE NOT NULL,  
  zustand_bei_rueckgabe VARCHAR2(20) NOT NULL CHECK (zustand_bei_rueckgabe IN  
( 'NEU', 'GUT', 'AKZEPTABEL', 'SCHLECHT' ))  
);
```

```
SELECT r.rueckgabedatum, r.zustand_bei_rueckgabe,  
       n.vorname || ' ' || n.nachname AS leiher  
FROM   rueckgabe r  
JOIN   ausleihe a ON a.ausleihe_id = r.ausleihe_id  
JOIN   ausleihanfrage an ON an.anfrage_id = a.anfrage_id  
JOIN   nutzer n ON n.nutzer_id = an.leiher_id  
WHERE  r.exemplar_id = 5  
ORDER BY r.rueckgabedatum;
```



	RUECKGABEDATUM	ZUSTAND_BEI_RUECKGABE	LEIHER
1	10.02.26	GUT	Clara Schneider
2	15.03.26	GUT	Hans Becker
3	28.04.26	AKZEPTABEL	Lars Koch

Rückgaben

Ein Rückgabeeintrag bestätigt, dass ein Exemplar zurückgegeben wurde und hält den Zustand zum Rückgabezeitpunkt fest.

Die Abfrage zeigt die Rückgabehistorie eines Exemplars über mehrere Ausleihen.

Tabelle: Anfrage_nachricht

```
CREATE SEQUENCE seq_anfrage_nachricht START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE anfrage_nachricht (  
  nachricht_id  NUMBER      DEFAULT seq_anfrage_nachricht.NEXTVAL PRIMARY KEY,  
  anfrage_id    NUMBER      NOT NULL REFERENCES ausleihanfrage(anfrage_id),  
  sender_id     NUMBER      NOT NULL REFERENCES nutzer(nutzer_id),  
  inhalt        VARCHAR2(2000) NOT NULL,  
  gesendet_am   DATE        DEFAULT SYSDATE NOT NULL,  
  gelesen       NUMBER(1)    DEFAULT 0 NOT NULL CHECK (gelesen IN (0,1))  
);
```

```
SELECT n.nachricht_id, n.gesendet_am, n.inhalt,  
       s.vorname || ' ' || s.nachname AS absender,  
       n.anfrage_id  
FROM anfrage_nachricht n  
JOIN nutzer s ON s.nutzer_id = n.sender_id  
WHERE n.gelesen = 0  
AND n.anfrage_id IN (  
  SELECT anfrage_id FROM ausleihanfrage  
  WHERE verleih_id = 25 OR leiher_id = 25  
)  
ORDER BY n.gesendet_am;
```



	⚡ NACHRICHT_ID	⚡ GESENDET_AM	⚡ INHALT	⚡ ABSENDER	⚡ ANFRAGE_ID
1	5007	20.03.26	Ich hätte Interesse.	Jan Hoffmann	2010
2	5013	05.05.26	Wann wäre eine Übergabe möglich?	Robert Neumann	2014
3	5014	06.05.26	Am Wochenende hätte ich Zeit.	Yvonne Schmitt	2014

Chat-Nachrichten

Die Nachrichten-Tabelle dient der Kommunikation zwischen Leiher und Verleiher während des Ausleihprozesses.

Die Abfrage zeigt den Posteingang des Benutzers mit der Id 25, also die ungelesenen Nachrichten aus seinen Anfragen.

Tabelle: Bewertung

```
CREATE SEQUENCE seq_bewertung START WITH 1000 INCREMENT BY 1 NOCACHE;
```

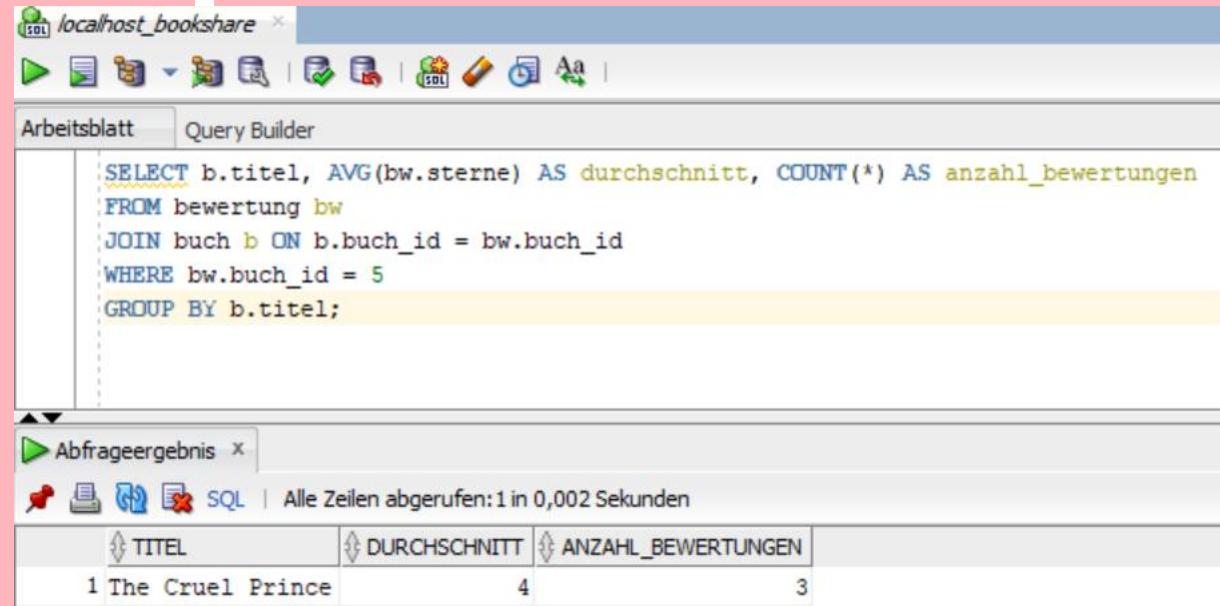
```
CREATE TABLE bewertung (  
  bewertung_id      NUMBER      DEFAULT seq_bewertung.NEXTVAL PRIMARY KEY,  
  ausleihe_id       NUMBER      NOT NULL REFERENCES ausleihe(ausleihe_id),  
  bewerter_id       NUMBER      NOT NULL REFERENCES nutzer(nutzer_id),  
  bewerteter_nutzer_id NUMBER      REFERENCES nutzer(nutzer_id), -- nullable:  
  Nutzerbewertung  
  buch_id           NUMBER      REFERENCES buch(buch_id), -- nullable: Buchbewertung  
  sterne            NUMBER(1)    NOT NULL CHECK (sterne BETWEEN 1 AND 5),  
  kommentar         VARCHAR2(2000),  
  datum            DATE          DEFAULT SYSDATE NOT NULL,  
  -- Entweder Buch oder Nutzer wird bewertet, nicht beides  
  CONSTRAINT chk_bewertung_ziel CHECK (  
    (buch_id IS NOT NULL AND bewerteter_nutzer_id IS NULL) OR  
    (buch_id IS NULL AND bewerteter_nutzer_id IS NOT NULL)  
  )  
);
```

```
SELECT b.titel, AVG(bw.sterne) AS durchschnitt, COUNT(*) AS anzahl_bewertungen  
FROM bewertung bw  
JOIN buch b ON b.buch_id = bw.buch_id  
WHERE bw.buch_id = 5  
GROUP BY b.titel;
```

Bewertungen

Eine Bewertung bezieht sich entweder auf ein Buch oder auf einen Nutzer, nie auf beides.

Die Abfrage zeigt die Durchschnittsbewertung und Anzahl der Bewertungen für das Buch mit der Id 5 „The Cruel Prince“.



The screenshot shows the SQL Developer interface. The top pane displays the following SQL query:

```
SELECT b.titel, AVG(bw.sterne) AS durchschnitt, COUNT(*) AS anzahl_bewertungen  
FROM bewertung bw  
JOIN buch b ON b.buch_id = bw.buch_id  
WHERE bw.buch_id = 5  
GROUP BY b.titel;
```

The bottom pane shows the query results in a table with the following data:

TITEL	DURCHSCHNITT	ANZAHL_BEWERTUNGEN
1 The Cruel Prince	4	3

Tabelle: Nutzer

```
CREATE SEQUENCE seq_nutzer START WITH 1000 INCREMENT BY 1 NOCACHE;

CREATE TABLE nutzer (
  nutzer_id      NUMBER      DEFAULT seq_nutzer.NEXTVAL PRIMARY KEY,
  vorname        VARCHAR2(100) NOT NULL,
  nachname       VARCHAR2(100) NOT NULL,
  email          VARCHAR2(255) NOT NULL UNIQUE,
  passwort_hash  VARCHAR2(255) NOT NULL,
  telefon        VARCHAR2(50),
  whatsapp       VARCHAR2(50),
  email_token     VARCHAR2(100),
  email_bestaetigt NUMBER(1)  DEFAULT 0 NOT NULL CHECK (email_bestaetigt IN (0,1)),
  suspended      NUMBER(1)  DEFAULT 0 NOT NULL CHECK (suspended IN (0,1)),
  deleted        NUMBER(1)  DEFAULT 0 NOT NULL CHECK (deleted IN (0,1)),
  created_at     DATE        DEFAULT SYSDATE NOT NULL,
  last_login     DATE,
  profilbild     BLOB
);

INSERT INTO nutzer (vorname, nachname, email, passwort_hash, telefon, whatsapp,
email_bestaetigt) VALUES ('Anna', 'Müller', 'anna.mueller@example.de', 'hash_001',
'0211 123456', '+49 211 123456', 1);

Select * from nutzer where nutzer_id < 11;
```

Benutzer

Die Nutzer-Tabelle enthält registrierte Plattformnutzer mit Authentifizierungs- und Statusdaten.

Alle Geschäftsprozesse hängen an dieser Tabelle. Ein Nutzer nimmt in unterschiedlichen Vorgängen unterschiedliche fachliche Rollen ein: Verleiher, Leiher, Bewerter, Melder.

Die Statements zeigen das Anlegen eines Benutzers und die Abfrage der ersten zehn Benutzer.

⚡	NUTZER_ID	VORNAME	NACHNAME	EMAIL	PASSWORT_HASH	TELEFON	WHATSAPP	EMAIL_TOKEN	EMAIL_BESTAETIGT	SUSPENDED	DELETED	CREATED_AT	LAST_LOGIN	PROFILBILD
1	1	Anna	Müller	anna.mueller@example.de	hash_001	0211 123456	+49 211 123456	(null)	1	0	0	31.05.26	(null)	(null)
2	2	Ben	Schmidt	ben.schmidt@example.de	hash_002	030 987654	+49 30 987654	(null)	1	0	0	31.05.26	(null)	(null)
3	3	Clara	Schneider	clara.schneider@example.de	hash_003	089 456789	(null)	(null)	1	0	0	31.05.26	(null)	(null)
4	4	David	Fischer	david.fischer@example.de	hash_004	(null)	+49 40 112233	(null)	1	0	0	31.05.26	(null)	(null)
5	5	Eva	Weber	eva.weber@example.de	hash_005	0221 334455	+49 221 334455	(null)	1	0	0	31.05.26	(null)	(null)
6	6	Felix	Meyer	felix.meyer@example.de	hash_006	0711 667788	(null)	(null)	1	0	0	31.05.26	(null)	(null)
7	7	Greta	Wagner	greta.wagner@example.de	hash_007	0351 998877	+49 351 998877	(null)	1	0	0	31.05.26	(null)	(null)
8	8	Hans	Becker	hans.becker@example.de	hash_008	(null)	+49 69 445566	(null)	1	0	0	31.05.26	(null)	(null)
9	9	Ina	Schulz	ina.schulz@example.de	hash_009	0931 556677	+49 931 556677	(null)	1	0	0	31.05.26	(null)	(null)
10	10	Jan	Hoffmann	jan.hoffmann@example.de	hash_010	0341 778899	(null)	(null)	1	0	0	31.05.26	(null)	(null)

Tabelle: Adresse

```
CREATE SEQUENCE seq_adresse START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE adresse (  
  adresse_id      NUMBER      DEFAULT seq_adresse.NEXTVAL PRIMARY KEY,  
  nutzer_id       NUMBER      NOT NULL REFERENCES nutzer(nutzer_id),  
  strasse         VARCHAR2(100) NOT NULL,  
  hausnummer      VARCHAR2(10) NOT NULL,  
  plz             VARCHAR2(10) NOT NULL,  
  stadt           VARCHAR2(100) NOT NULL,  
  land            VARCHAR2(100) DEFAULT 'Deutschland' NOT NULL,  
  breitengrad     NUMBER(9,6), -- Latitude  
  laengengrad     NUMBER(9,6), -- Longitude  
  ist_rechnungsadresse NUMBER(1) DEFAULT 0 NOT NULL CHECK (ist_rechnungsadresse  
IN (0,1)),  
  ist_standard    NUMBER(1)   DEFAULT 0 NOT NULL CHECK (ist_standard IN (0,1))  
);
```

```
WITH ergebnis AS (  
  SELECT b.titel,  
    n.vorname || ' ' || n.nachname AS besitzer,  
    a.stadt,  
    a.strasse || ' ' || a.hausnummer AS abholort,  
    ROUND(6371 * ACOS(  
      COS(51.5142 * 3.14159/180) * COS(a.breitengrad * 3.14159/180)  
      * COS((a.laengengrad - 7.4684) * 3.14159/180)  
      + SIN(51.5142 * 3.14159/180) * SIN(a.breitengrad * 3.14159/180)  
    ), 1) AS entfernung_km  
  FROM  exemplar e  
  JOIN  adresse a ON a.adresse_id = e.abholort_id  
  JOIN  buch b   ON b.buch_id   = e.buch_id  
  JOIN  nutzer n ON n.nutzer_id = e.besitzer_id  
  WHERE e.gesperrt = 0  
  AND   a.breitengrad IS NOT NULL  
)  
SELECT *  
FROM  ergebnis  
WHERE entfernung_km <= 35  
ORDER BY entfernung_km;
```



Adressen

Jeder Nutzer kann mehrere Adressen hinterlegen.

Breitengrad und Längengrad unterstützen die räumliche Suche nach Exemplaren.

Die Query zeigt verfügbare Exemplare im Umkreis eines Standorts, berechnet über die Haversine-Formel.

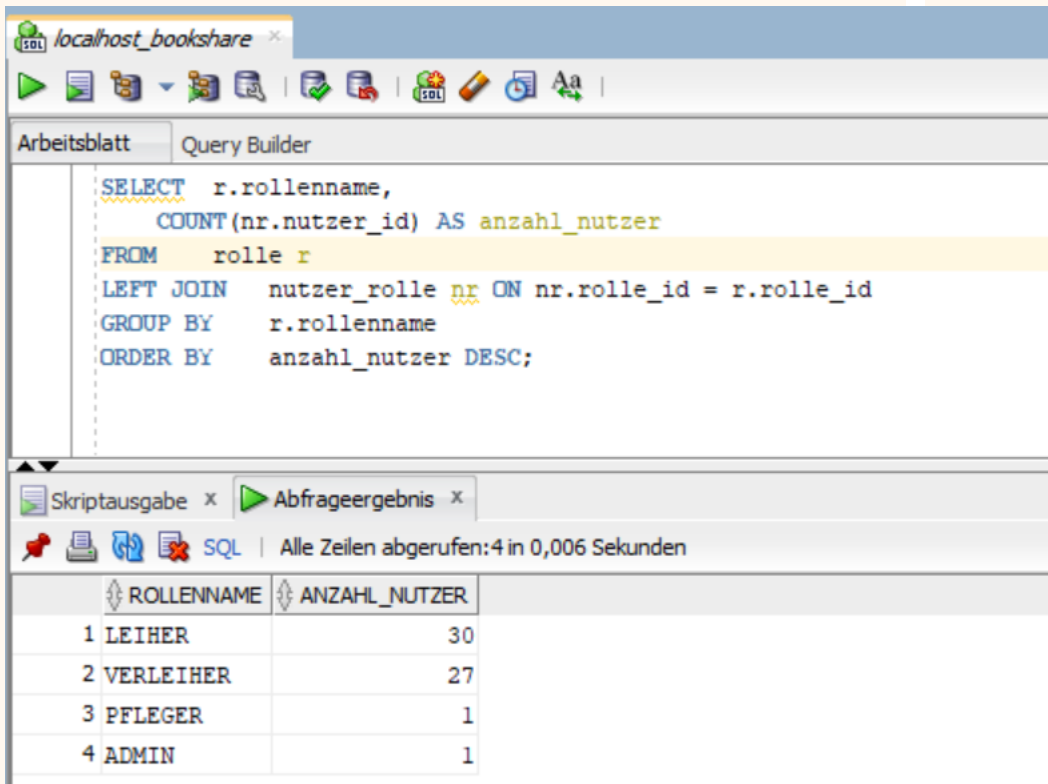
TITEL	BESITZER	STADT	ABHOLORT	ENTFERNUNG_KM
1 Pride and Prejudice	Lars Koch	Dortmund	Bergstraße 6	0
2 The Summer I Turned Pretty Trilogy	Lars Koch	Dortmund	Bergstraße 6	0
3 The Sun Also Rises	Lars Koch	Dortmund	Bergstraße 6	0
4 Misery	Lars Koch	Dortmund	Bergstraße 6	0
5 The Fellowship of the Ring	Lars Koch	Dortmund	Bergstraße 6	0
6 Exile	Lars Koch	Dortmund	Bergstraße 6	0
7 The Great Hunt (The Wheel of Time Book 2)	Queenie Schröder	Gelsenkirchen	Kastanienallee 12	26,5
8 Red, White & Royal Blue	Queenie Schröder	Gelsenkirchen	Kastanienallee 12	26,5
9 The Lies of Locke Lamora	Queenie Schröder	Gelsenkirchen	Kastanienallee 12	26,5
10 Voyage au Centre de la Terre	Queenie Schröder	Gelsenkirchen	Kastanienallee 12	26,5
11 Anthem	Queenie Schröder	Gelsenkirchen	Kastanienallee 12	26,5
12 Double Down	Queenie Schröder	Gelsenkirchen	Kastanienallee 12	26,5
13 Dork Diaries Collection	Maria Bauer	Essen	Parkweg 17	32,3
14 A Good Girl's Guide to Murder	Maria Bauer	Essen	Parkweg 17	32,3
15 Voyage au Centre de la Terre	Maria Bauer	Essen	Parkweg 17	32,3
16 Wolves of the Calla	Maria Bauer	Essen	Parkweg 17	32,3
17 Deception Point	Maria Bauer	Essen	Parkweg 17	32,3
18 We Were Never Meant To Be	Maria Bauer	Essen	Parkweg 17	32,3

Tabelle: Rolle

```
CREATE SEQUENCE seq_rolle START WITH 1000 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE rolle (  
  rolle_id NUMBER DEFAULT seq_rolle.NEXTVAL PRIMARY KEY,  
  rollenname VARCHAR2(50) NOT NULL UNIQUE  
);
```

```
SELECT      r.rollenname,  
            COUNT(nr.nutzer_id)          AS anzahl_nutzer  
  
FROM        rolle r  
LEFT JOIN   nutzer_rolle nr ON nr.rolle_id = r.rolle_id  
GROUP BY   r.rollenname  
ORDER BY   anzahl_nutzer DESC;
```



The screenshot shows the SQL Developer interface. The top toolbar includes icons for running queries, saving, and other database functions. The 'Arbeitsblatt' (Worksheet) tab is active, displaying the SQL query. Below the query, the 'Skriptausgabe' (Script Output) and 'Abfrageergebnis' (Query Result) tabs are visible. The 'Abfrageergebnis' tab shows the results of the query in a table format.

	ROLLENNAME	ANZAHL_NUTZER
1	LEIHER	30
2	VERLEIHER	27
3	PFLEGER	1
4	ADMIN	1

Rollen

Hinterlegt sind vier Berechtigungsrollen des Systems: LEIHER, VERLEIHER, PFLEGER und ADMIN.

Jede Rolle steht genau einmal in der Tabelle.

Nutzer erhalten Rollen über die Zuordnungstabelle NUTZER_ROLLE.

Die Vergabe von Rechten je Rolle soll zur Anwendungslogik gehören und ist nicht Teil des Datenmodells.

Die Abfrage zeigt, wie viele Nutzer jede Rolle aktuell tragen: Jeder Benutzer ist „Leiher“, aber es gibt beispielsweise nur einen (Stammdaten-) Pfleger.

Tabelle: Nutzer_rolle

```
CREATE TABLE nutzer_rolle (  
  nutzer_id NUMBER NOT NULL REFERENCES nutzer(nutzer_id),  
  rolle_id NUMBER NOT NULL REFERENCES rolle(rolle_id),  
  vergeben_am DATE DEFAULT SYSDATE NOT NULL,  
  CONSTRAINT pk_nutzer_rolle PRIMARY KEY (nutzer_id, rolle_id)  
);
```

```
SELECT n.vorname || ' ' || n.nachname AS nutzer,  
       COUNT(nr.rolle_id) AS anzahl_rollen  
FROM   nutzer_rolle nr  
JOIN   nutzer n ON n.nutzer_id = nr.nutzer_id  
GROUP BY n.vorname, n.nachname  
HAVING COUNT(nr.rolle_id) > 1  
ORDER BY anzahl_rollen DESC;
```



NUTZER	ANZAHL_ROLLEN
1 Anna Müller	2
2 Dirk Köhler	2
3 Clara Schneider	2
4 David Fischer	2
5 Eva Weber	2
6 Felix Meyer	2
7 Greta Wagner	2
8 Hans Becker	2
9 Ina Schulz	2
10 Jan Hoffmann	2
11 Karin Schäfer	2
12 Lars Koch	2
13 Maria Bauer	2
14 Nico Richter	2
15 Olga Klein	2
16 Paul Wolf	2
17 Queenie Schröder	2
18 Robert Neumann	2
19 Sara Schwarz	2
20 Thomas Zimmermann	2
21 Ursula Braun	2
22 Viktor Krüger	2
23 Wendy Hartmann	2
24 Xaver Lange	2
25 Yvonne Schmitt	2
26 Zacharias Werner	2
27 Alma Peters	2
28 Celine Lehmann	2
29 Ben Schmidt	2

Nutzer-Rollen-Zuordnung

Benutzern werden eine oder mehrere Rollen zugeordnet. Die Kreuztabelle bildet eine n:m-Beziehung zwischen NUTZER und ROLLE ab.

So kann ein Nutzer gleichzeitig Verleiher und Leiher sein, ohne dass seine Benutzerdaten redundant gehalten werden.

Die Abfrage zeigt alle Nutzer mit mehr als einer Rolle. Nur „Bruno Krause“ ist Leiher, Verleiher und Pfleger und hat mehr als 2 Rollen.

3. Zusammenfassung

Aspekte der Umsetzung

Zusammenfassung

Datenbankumgebung

Als DBMS wurde Oracle Database 23ai Free gewählt, betrieben als Container unter Podman. Die Ausführung und der Test der SQL-Statements erfolgten mit Oracle SQL Developer. Das ER-Diagramm wurde mit DBeaver erstellt.

Datenbankschema

Das Schema umfasst 16 Tabellen in dritter Normalform. Primärschlüssel werden über Sequences vergeben. Für relevante Felder wurden Wertebereichsprüfungen als CHECK-Constraints auf Datenbankebene hinterlegt.

SQL-Dateien

Die Installation ist vollständig skriptgestützt. `create_tables.sql` legt das Schema reproduzierbar an.

Bücher und Autoren wurden über die Open Library Search API abgerufen und per Python-Skript aufbereitet. Herkunftsland und Beschreibung der Autoren wurden über Wikidata ergänzt. Die Rolle-Tabelle enthält als einzige Tabelle weniger als zehn Einträge, da das System vier Rollen vorsehen würde. Das Ergebnis ist in `dummy_data.sql` hinterlegt.

`queries.sql` enthält alle Abfragen als eigenständig ausführbare Statements. Jede Abfrage deckt einen Usecase ab, der in einer Anwendung implementiert werden würde. Enthalten sind einfache SELECTs ebenso wie Common Table Expressions (With-Clause) und Abfragen mit Subqueries.

Ausblick

Die Tabellen sind vier fachlichen Bereichen zugeordnet: Katalog, Ausleihprozess, Bewertungen und Benutzer. In einer produktiven Umsetzung könnte man diese Bereiche als eigenständige Dienste mit separater Datenhaltung realisieren. Das Schema enthält Felder für Passwort-Hash und E-Mail-Token. In einer produktiven Umsetzung würde man die Authentifizierung und ggf. die Autorisierung an einen externen Identity-Provider auslagern und diese Felder entfernen.

Das Modell ließe sich um weitere Funktionen erweitern. Eine Streitfall-Tabelle zur Dokumentation von Konflikten zwischen Leiher und Verleiher wäre ein naheliegender nächster Schritt. Mit anderen Erweiterungen wie Zahlungsabwicklung, einem Werbesystem oder einem Logistik-Modul wäre ein Schema mit 30 oder mehr Tabellen denkbar.

Entwurfsentscheidungen

Gründe für die Wahl von Oracle als DBMS

- Per Docker-Image läuft die Datenbank lokal, ohne Installation.
- Die Free Release-Version hat denselben Funktionsumfang wie die Enterprise-Edition.
- Es gibt eine breite Tool-Unterstützung: SQL Developer, DBeaver, Python (oracledb).
- Oracle steht im DB-Engines Ranking auf Platz 1.

Trennung von Buch und Exemplar

- Buchmetadaten (Titel, Autor, Genre) liegen einmalig in BUCH.
- Physische Exemplare mit Zustand, Besitzer und Abholort liegen in EXEMPLAR.
- Mehrere Nutzer können dasselbe Buch im Bestand haben, ohne Metadaten zu duplizieren.
- Die Suche nach verfügbaren Exemplaren eines Titels ist dadurch eindeutig.

CHECK-Constraints

Für relevante Felder wurden Wertebereichsprüfungen auf Datenbankebene hinterlegt.

Eine Erweiterung des Wertebereichs muss sowohl in der Anwendung als auch im Datenbankschema durchgeführt werden.

Allerdings ist die Datenintegrität auf diese Art sowohl bei Fehlern in der Anwendungslogik als auch bei externen Buchimporten gesichert.

Hinweis zur Präsentation

Die vier Kreuztabellen sind im ERM keine eigenständigen Entitäten.

Sie erhalten dennoch jeweils eine eigene Folie, da sie eigene DDL-Statements und Abfragen haben.

Literaturverzeichnis

Open Library. (n.d.). *Open Library*. <https://openlibrary.org>

Wikimedia Foundation. (n.d.). *Wikidata*. <https://www.wikidata.org>